Name- Aditya

Roll no.- 2401010134

Course- B.tech CSE Core (Section C)

Subject- Analysis and design of Algorithms Lab

Assignment 3

Task 1: Graph Representation

```
# Graph Representation graph
= {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': ['F'],
   'F': []
}
# Adjacency Matrix Representation  nodes
= ['A','B','C','D','E','F'] matrix =
[[0]*len(nodes) for _ in range(len(nodes))]

for i, node in enumerate(nodes):
for neighbor in graph[node]:        j
= nodes.index(neighbor)
matrix[i][j] = 1

print("Adjacency Matrix:") for
row in matrix:
print(row)
```

Output:

```
Adjacency Matrix:
[0, 1, 1, 0, 0, 0]
[0, 0, 0, 1, 1, 0]
[0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 0]
```

Task 2: BFS

```
from collections import deque


def bfs(graph, start):     visited
= set()     queue =
deque([start])     while queue:
        node = queue.popleft()        if
node not in visited:            print(node,
end=" ")            visited.add(node)
queue.extend(graph[node]) print("BFS
Traversal:") bfs(graph,
'A')
```

Output:

```
BFS Traversal:
A B C D E F
```

Task 3: DFS

```
def dfs(graph, node, visited=set()): if node not in visited:
print(node, end=" ")        visited.add(node)        for neighbor
in graph[node]:                    dfs(graph, neighbor, visited)
print("\nDFS Traversal:") dfs(graph, 'A')
```

Output:

```
DFS Traversal:
A B D E F C
```

Task 4: Topological Sort

```
from collections import defaultdict


def topological_sort(graph):
    visited = set()     stack
= []


    def dfs(node):
visited.add(node)        for
neighbor in graph[node]:            if
neighbor not in visited:
                dfs(neighbor)
stack.append(node)     for
node in graph:        if node not
in visited:
dfs(node) print("Topological
Sort:") print(stack[::-1])



topological_sort(graph)
```

Output:

```
Topological Sort:
['A', 'C', 'B', 'E', 'F', 'D']
```

Task 5: Dijkstra Algorithm

```
import heapq  def dijkstra(graph, start):
distances = {node: float('inf') for node in graph}
distances[start] =
0     pq = [(0, start)]
```

```
    while pq:
        current_distance, node = heapq.heappop(pq)

        for neighbor, weight in graph[node].items():
distance = current_distance + weight

            if distance < distances[neighbor]:
distances[neighbor] = distance
heapq.heappush(pq, (distance, neighbor))

    return distances

graph_dijkstra = {
    'A': {'B': 4, 'C': 2},
    'B': {'C': 5, 'D': 10},
    'C': {'E': 3},
    'D': {'F': 11},
    'E': {'D': 4},
    'F': {}
}

print("Dijkstra Shortest Paths:") print(dijkstra(graph_dijkstra,
'A'))
```

Output:

```
Dijkstra Shortest Paths:
{'A': 0, 'B': 4, 'C': 2, 'D': 9, 'E': 5, 'F': 20}
```

Task 6: Bellman Ford

```
def bellman_ford(edges, vertices, start):
```

```
dist = {v: float('inf') for v in vertices}     dist[start]
= 0


   for _ in range(len(vertices)-1):
for u, v, w in edges:            if
dist[u] + w < dist[v]:
dist[v] = dist[u] + w     return
dist edges = [
   ('A','B',4),
   ('A','C',5),
   ('B','C',-3),
   ('C','D',4)
]
vertices = ['A','B','C','D']


print("Bellman Ford:") print(bellman_ford(edges,
vertices, 'A'))
```

Output:

```
Bellman Ford:
{'A': 0, 'B': 4, 'C': 1, 'D': 5}
```

Task 7: Minimum Spanning Tree

```
import heapq


def prim(graph, start):
visited = set()     min_heap
= [(0, start)]     total_cost
= 0     while min_heap:
        cost, node = heapq.heappop(min_heap)
```

```
    if node not in visited:          visited.add(node)
total_cost
+= cost


        for neighbor, weight in graph[node].items():
if neighbor not in visited:
                heapq.heappush(min_heap, (weight, neighbor))
return total_cost graph_prim = {
   'A': {'B': 2, 'C': 3},
   'B': {'A': 2, 'C': 1, 'D': 4},
   'C': {'A': 3, 'B': 1, 'D': 5},
   'D': {'B': 4, 'C': 5}
}
print("Prim MST Cost:") print(prim(graph_prim,
'A'))
```

Output:

```
Prim MST Cost:
7
```

Task 8: Kruskal Algorithm:

```
def kruskal(edges, vertices):
parent = {v:v for v in vertices}     def
find(v):        if parent[v] != v:
parent[v] = find(parent[v])
return parent[v]     mst = []
edges.sort(key=lambda x: x[2])     for
u,v,w in edges:        root1 = find(u)
root2 = find(v)
```

```
        if root1 != root2:
mst.append((u,v,w))
parent[root1] = root2     return
mst edges = [
    ('A','B',2),
    ('A','C',3),
    ('B','C',1),
    ('B','D',4),
    ('C','D',5)
] vertices = ['A','B','C','D'] print("Kruskal
MST:") print(kruskal(edges, vertices))
```

Output:

```
Kruskal MST:
[('B', 'C', 1), ('A', 'B', 2), ('B', 'D', 4)]
```

Github; adiipawarrrr/-algorithms_assignment